

Spawning Interactive Shells

At the end of the last section, we established a shell session with the target. Initially, our shell was limited (sometimes referred to as a jail shell), so we used python to spawn a TTY bourne shell to give us access to more commands and a prompt to work from. This will likely be a situation we find ourselves in more and more as we practice on Hack The Box and in the real world on engagements.

There may be times that we land on a system with a limited shell, and Python is not installed. In these cases, it's good to know that we could use several different methods to spawn an interactive shell. Let's examine some of them.

Know that whenever we see `/bin/sh` or `/bin/bash`, this could also be replaced with the binary associated with the shell interpreter language present on that system. With most Linux systems, we will likely come across `bourne shell (/bin/sh)` and `bourne again shell (/bin/bash)` present on the system natively.

`/bin/sh -i`

This command will execute the shell interpreter specified in the path in interactive mode (`-i`).

Interactive

Interactive

```
/bin/sh -i
sh: no job control in this shell
sh-4.2$
```

Perl

If the programming language `Perl` is present on the system, these commands will execute the shell interpreter specified.

Perl To Shell

Perl To Shell

```
perl -e 'exec "/bin/sh";'
```

Perl To Shell

```
perl: exec "/bin/sh";
```

The command directly above should be run from a script.

Ruby

If the programming language `Ruby` is present on the system, this command will execute the shell interpreter specified:

Ruby To Shell

Ruby To Shell

```
ruby: exec "/bin/sh"
```

The command directly above should be run from a script.

Lua

If the programming language [Lua](#) is present on the system, we can use the `os.execute` method to execute the shell interpreter specified using the full command below:

Lua To Shell

Lua To Shell

```
lua: os.execute('/bin/sh')
```

The command directly above should be run from a script.

AWK

[AWK](#) is a C-like pattern scanning and processing language present on most UNIX/Linux-based systems, widely used by developers and sysadmins to generate reports. It can also be used to spawn an interactive shell. This is shown in the short awk script below:

AWK To Shell

AWK To Shell

```
awk 'BEGIN {system("/bin/sh")}'
```

Find

[Find](#) is a command present on most Unix/Linux systems widely used to search for & through files and directories using various criteria. It can also be used to execute applications and invoke a shell interpreter.

Using Find For A Shell

Using Find For A Shell

```
find / -name nameoffile -exec /bin/awk 'BEGIN {system("/bin/sh")}' \;
```

This use of the find command is searching for any file listed after the `-name` option, then it executes `awk (/bin/awk)` and runs the same script we discussed in the awk section to execute a shell interpreter.

Using Exec To Launch A Shell

Using Find For A Shell

```
find . -exec /bin/sh \; -quit
```

This use of the find command uses the execute option (**-exec**) to initiate the shell interpreter directly. If **find** can't find the specified file, then no shell will be attained.

VIM

Yes, we can set the shell interpreter language from within the popular command-line-based text-editor **VIM**. This is a very niche situation we would find ourselves in to need to use this method, but it is good to know just in case.

Vim To Shell

Vim To Shell

```
vim -c '!/bin/sh'
```

Vim Escape

Vim Escape

```
vim
:set shell=/bin/sh
:shell
```

Execution Permissions Considerations

In addition to knowing about all the options listed above, we should be mindful of the permissions we have with the shell session's account. We can always attempt to run this command to list the file properties and permissions our account has over any given file or binary:

Permissions

Permissions

```
ls -la <path/to/fileorbinary>
```

We can also attempt to run this command to check what **sudo** permissions the account we landed on has:

Sudo -l

Sudo -l

```
sudo -l
Matching Defaults entries for apache on ILF-WebSrv:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User apache may run the following commands on ILF-WebSrv:
    (ALL : ALL) NOPASSWD: ALL
```

The **sudo -l** command above will need a stable interactive shell to run. If you are not in a full shell or sitting in an unstable shell, you may not get any return from it. Not only will considering permissions allow us to see what commands we can execute, but it may also start to give us an idea of potential vectors that will allow us to escalate privileges.

[← Previous](#)[Next →](#)[✔ Mark Complete & Next](#) [Cheat Sheet](#)

Table of Contents

Introduction

Shells Jack Us In, Payloads Deliver Us Shells



CAT5 Security's Engagement Preparation



Shell Basics

 Anatomy of a Shell



 Bind Shells



 Reverse Shells



Payloads

Introduction to Payloads



 Automating Payloads & Delivery with Metasploit

Crafting payloads with MSFvenom



Windows Shells

 Infiltrating Windows



NIX Shells

 Infiltrating Unix/Linux



[Spawning Interactive Shells](#)

Web Shells

Introduction to Web Shells

 Laudanum, One Webshell to Rule Them All

 Antak Webshell

 PHP Web Shells

Skills Assessment

 The Live Engagement

Additional Considerations

Detection & Prevention

My Workstation

OFFLINE

 [Start Instance](#)

 Start instance

 / 1 spawns left